

Կառուցվածքային տիպեր C++ լեզվում

Թելադրվող համառոտ դասախոսական նյութ

Այս նյութը ներկայացնում է կառուցվածքների կիրառման հիմնական կանոնները՝ կառուցվածքի հայտարարումից մինչև զանգվածներ, ցուցիչներ և ֆունկցիաներին փոխանցում: Նպատակն է տարբեր տիպերի փոխկապակցված տվյալները դիտարկել որպես մեկ ամբողջական գրառում և ընտրել դրանց մշակման ճիշտ եղանակը:

1 Կառուցվածքային տիպի գաղափարը

Կառուցվածքը օգտագործողի կողմից սահմանվող տվյալների տիպ է, որը մեկ անվան տակ միավորում է տարբեր տիպերի, բայց տրամաբանորեն կապված տվյալներ: Կառուցվածքը հարմար է որևէ իրական օբյեկտի կամ երևույթի նկարագրության համար: Օրինակ՝ ուսանողի անունը, տարիքը և գնահատականը տարբեր տիպերի արժեքներ են, սակայն միասին ներկայացնում են մեկ ուսանողի գրառում:

Կառուցվածքի սահմանումը նկարագրում է ապագա գրառման կազմը: Սահմանման պահին հիշողության մեջ տվյալներ չեն ստեղծվում: Հիշողություն հատկացվում է միայն այն ժամանակ, երբ տվյալ կառուցվածքային տիպով հայտարարվում է փոփոխական:

```
struct Student {  
    string name;  
    int age;  
    double grade;  
};
```

2 Կառուցվածքային փոփոխականներ և անդամներ

Կառուցվածքի ներսում հայտարարված փոփոխականները կոչվում են անդամներ կամ դաշտեր: Կառուցվածքային փոփոխականը պարունակում է կառուցվածքում նկարագրված բոլոր անդամները: Սովորական կառուցվածքային փոփոխականի անդամին դիմելու համար օգտագործվում է կետային գործողությունը:

```
Student student;  
student.name = "Ani";  
student.age = 18;  
student.grade = 16.5;
```

Կառուցվածքային փոփոխականը կարելի է սկզբնաբանել հայտարարման պահին: Արժեքները պետք է համապատասխանեն անդամների հայտարարման հերթականությանը և տիպերին: Յուրաքանչյուր փոփոխական ունի իր սեփական անդամները, ուստի մեկ գրառման փոփոխությունը չի փոխում մյուս գրառումը:

```
Student student = {"Ani", 18, 16.5};
```

3 Կառուցվածքների զանգվածներ

Մինևույն կառուցվածքային տիպին պատկանող բազմաթիվ գրառումներ կարելի է պահել զանգվածում: Չանգվածի յուրաքանչյուր տարր ամբողջական կառուցվածքային փոփոխական է: Նախ ընտրվում է զանգվածի տարրը, ապա կետային գործողությամբ՝ այդ տարրի անհրաժեշտ անդամը:

```
Student students[3];
students[0].name = "Ani";
students[0].grade = 16.5;
```

Կառուցվածքների զանգվածը մշակվում է ցիկլով: Նույն ցիկլում կարելի է մուտքագրել, արտածել, որոնել, հաշվել կամ համեմատել գրառումների անդամները: Որոնման արդյունքը հաճախ պահվում է ինդեքսի միջոցով, որովհետև ինդեքսը թույլ է տալիս հետագայում դիմել տվյալ գրառման բոլոր անդամներին:

4 Կառուցվածքի ցուցիչ

Կառուցվածքի ցուցիչը պահում է կառուցվածքային փոփոխականի հասցեն: Ցուցիչը հայտարարվում է կառուցվածքի տիպով, իսկ հասցեն ստացվում է հասցեի գործողությամբ: Եթե ցուցիչը մատնանշում է կառուցվածք, ապա անդամներին դիմելու համար հիմնականում օգտագործվում է սլաք գործողությունը:

```
Student student = {"Ani", 18, 16.5};
Student* ptr = &student;
cout << ptr->name;
ptr->grade = 18;
```

`ptr->grade` գրառումը համարժեք է `(\*ptr).grade` գրառմանը: Սլաք գործողությունը նախ ստանում է ցուցիչով մատնանշված կառուցվածքը, ապա ընտրում է դրա անդամը: Ցուցիչը կիրառելուց առաջ պետք է համոզվել, որ այն պարունակում է վավեր հասցե և չի հավասարվում `nullptr` արժեքին:

5 Կառուցվածքի փոխանցումը \$ֆունկցիային

Արժեքով փոխանցման ժամանակ \$ֆունկցիան ստանում է կառուցվածքի պատճենը: \$ֆունկցիայի ներսում պատճենի փոփոխությունը չի ազդում սկզբնական գրառման վրա: Մեծ կառուցվածքների դեպքում արժեքով փոխանցումը կարող է նաև ավելորդ պատճենման ծախս առաջացնել:

```
void changeGrade(Student student) {
    student.grade = 20;
}
```

Հղումով փոխանցման ժամանակ \$ֆունկցիան աշխատում է սկզբնական կառուցվածքի հետ: Եթե պարամետրը հայտարարված է առանց `const`-ի, \$ֆունկցիան կարող է փոփոխել կառուցվածքի անդամները: Այս եղանակը հարմար է տվյալների մուտքագրման և փոփոխման համար:

```
void changeGrade(Student& student) {
    student.grade = 20;
}
```

Ցուցիչով փոխանցման ժամանակ \$ֆունկցիան ստանում է կառուցվածքի հասցեն: \$ֆունկցիայի կանչի ժամանակ փոխանցվում է փոփոխականի հասցեն, իսկ անդամներին դիմելու համար օգտագործվում է `->` գործողությունը: Այս եղանակը պահանջում է հասցեի վավերության ստուգում:

```
void changeGrade(Student* student) {
    if (student != nullptr)
        student->grade = 20;
}
```

Եթե ֆունկցիան պետք է միայն կարդա և արտածի տվյալները, նպատակահարմար է կառուցվածքը փոխանցել հաստատուն հղումով՝ `const Student&`: Այդ եղանակը չի ստեղծում պատճեն և միաժամանակ արգելում է ֆունկցիայի ներսում կառուցվածքի փոփոխությունը:

```
void printStudent(const Student& student) {  
    cout << student.name << " " << student.grade;  
}
```

6 Ընտրության հիմնական կանոնը

Փոխանցման ձև	Ազդեցությունը սկզբնական տվյալների վրա	Երբ կիրառել
Արժեքով	Չի փոխում	Երբ անհրաժեշտ է անկախ պատճեն
Հղումով	Կարող է փոխել	Մուտքագրում կամ փոփոխում
Հաստատուն հղումով	Չի փոխում	Արագ և անվտանգ ընթերցում
Ցուցիչով	Կարող է փոխել	Երբ աշխատում ենք հասցեներով կամ թույլատրվում է <code>nullptr</code>

7 Ամփոփում

Կառուցվածքը թույլ է տալիս տարբեր տիպերի փոխկապակցված տվյալները ներկայացնել մեկ ամբողջական գրառմամբ: Կառուցվածքների զանգվածը պահում է միատեսակ գրառումների հավաքածու, իսկ ցուցիչը՝ գրառման հասցեն: Ֆունկցիային փոխանցման եղանակը ընտրվում է ըստ նպատակի՝ պատճեն ստանալ, սկզբնական գրառումը փոփոխել կամ այն միայն կարդալ: